

WEB UYGULAMASINA GENEL BAKIŞ

[Kaynak](#)

İÇERİK

1. Web Application Overview (Web Uygulamalarına Genel Bakış)
2. Uniform Resource Locator (URL)
3. HTTP Messages (HTTP Mesajları)
4. HTTP Request (Request Line ve Metotlar)
5. HTTP Request - Headers (Başlıklar) ve Body (Gövde)
6. HTTP Response (Status Line ve Durum Kodları)
7. HTTP Response - Headers (Başlıklar) ve Body (Gövde)
8. Security Headers (Güvenlik Başlıkları)

Web Application Overview (Web Uygulamalarına Genel Bakış)

Bir web uygulamasını anlamak için onu bir gezegene benzetebiliriz. Astronotlar gezegenin yüzeyini keşfetmek için seyahat ederler; tıpkı bir kullanıcının web tarayıcısı (browser) aracılığıyla bir web uygulamasını gezmesi gibi. Biz sadece yüzeyi görsek de, yüzeyin altında devasa bir işleyiş vardır. Bu benzetmede gezegenin tamamı bir **Web Server** (Web Sunucusu) iken, bizim gördüğümüz kısım sadece web sayfalarıdır.

Bir web uygulamasını temel olarak iki ana yapıya ayırırız: **Front End** (Ön Yüz) ve **Back End** (Arka Yüz).

Front End (Ön Yüz)

Front End, gezegenin yüzeyi gibidir; bir astronotun doğa kanunları çerçevesinde görebildiği ve etkileşime girebildiği her şeydir. Kullanıcı, web uygulamasıyla etkileşime girmek için tarayıcısını kullanır ve bu aşamada üç temel teknoloji devreye girer: **HTML**, **CSS** ve **JavaScript**.

1. HTML (Hypertext Markup Language)

HTML, web uygulamalarının temel yapı taşıdır. Tarayıcıya neyi, nasıl görüntüleyeceğini söyleyen bir komut dizisi veya kod kümesidir.

- **Analoji:** HTML'i gezegendeki basit organizmaların **DNA**'sına benzetebiliriz. Bu DNA, organizmanın nasıl bir araya getirileceğine dair temel talimatları içerir.
- **Teknik Not:** HTML bir programlama dili değil, bir işaretleme dilidir. Sayfanın iskeletini (başlıklar, paragraflar, resim yerleşimleri) oluşturur.

2. CSS (Cascading Style Sheets)

CSS, web uygulamasının standart görünümünü (renkler, yazı tipleri, düzenler/layout) tanımlar.

- **Analoji:** DNA örneğinden devam edersek; CSS, DNA'nın o organizmanın rengini, şeklini, boyutunu ve dokusunu belirleyen kısımdır.
- **Önem:** Sayfayı görsel olarak şekillendirir. Eğer CSS olmasaydı, tüm web siteleri 90'lı yıllardan kalma düz metin belgeleri gibi görünürdü.

3. JS (JavaScript)

JavaScript, tarayıcı içinde daha karmaşık ve dinamik etkinliklere olanak tanıyan kısımdır. HTML sadece "neyin gösterileceği" ile ilgilenirken, JavaScript "karar verme" ve "etkileşim" süreçlerini yönetir.

- **Analoji:** JavaScript'i gelişmiş bir organizmanın **beyni** olarak düşünebiliriz. Çevreden gelen etkileşimlere göre kararlar verilmesini sağlar.
- **Önem:** Modern web uygulamalarında tıklanan butonların form göndermesi, sayfa yenilenmeden verilerin güncellenmesi veya animasyonların tetiklenmesi tamamen JS sayesinde olur.

Back End (Arka Yüz)

Back End, web tarayıcısı içinde doğrudan görmediğimiz ancak uygulamanın çalışması için hayati önem taşıyan kısımdır. Gezegen örneğinde bunlar; binaları ayakta tutan taşıyıcı kolonlar, soluduğumuz hava veya bizi yerde tutan yerçekimi gibidir.

1. Database (Veritabanı)

Bilgilerin depolandığı, değiştirildiği ve geri çağrıldığı (retrieval) merkezdir.

- **İşlev:** Kullanıcı tercihleri, giriş bilgileri veya profil verileri burada saklanır.
- **Analoji:** Gezegendeki sakinlerin konumları haritalarda tutması, günlük tutmaları veya kütüphanelerdeki dosyalar birer veritabanı örneğidir.

2. Infrastructure (Altyapı)

Web uygulamasının altında yatan donanımsal ve yazılımsal katmandır. Bu bileşenler şunları içerir:

- **Web Sunucuları (Web Servers):** İstekleri karşılayan ana yazılım.
- **Uygulama Sunucuları (Application Servers):** İş mantığını (business logic) yürüten kısım.
- **Storage (Depolama):** Dosyaların fiziksel olarak tutulduğu alan.
- **Networking Devices:** Verinin iletilmesini sağlayan router, switch gibi ağ cihazları.

- **Analoji:** Gezegendeki yollar, bu yollarda giden arabalar ve arabalara güç veren yakıt, altyapıyı temsil eder.

3. WAF (Web Application Firewall)

WAF (Web Uygulaması Güvenlik Duvarı), isteğe bağlı ancak kritik bir güvenlik bileşenidir. Web sunucusuna gelen trafiği denetleyerek tehlikeli veya kötü niyetli istekleri filtreler ve koruma sağlar.

- **Analoji:** Bir gezegenin atmosferinin, canlıları zararlı UV ışınlarından korumasına benzer.
- **Siber Güvenlik Notu:** Pentest (Sızma Testi) süreçlerinde karşılaşılan ilk engellerden biridir; payload'ların (zararlı yükler) sunucuya ulaşmadan engellenmesine neden olabilir.

Özet ve Mantık Zinciri

Bir web isteği gönderdiğinizde süreç şu şekilde işler:

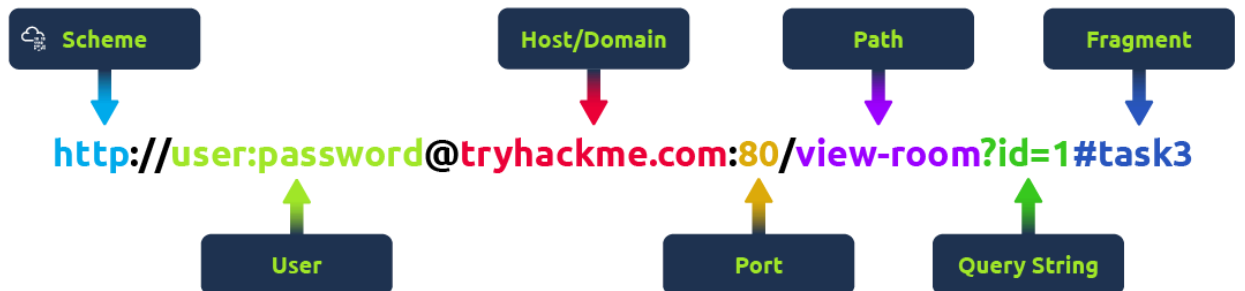
1. **Front End** katmanında (tarayıcıda) bir butona basarsınız.
2. **JavaScript** bu isteği işler ve **Altyapı** (Ağ cihazları) üzerinden sunucuya gönderir.
3. İstek **WAF** kontrolünden geçer; eğer zararlı değilse **Web Sunucusuna** ulaşır.
4. Sunucu, gerekirse **Veritabanından** bilgi çeker ve sonucu size **HTML/CSS** olarak geri döndürür.

Unutma: Front End "kullanıcı deneyimine", Back End ise "sistemin işleyişine ve güvenliğine" odaklanır. Bir siber güvenlik uzmanı olarak her iki tarafın zafiyetlerini (Örn: Front End'de XSS, Back End'de SQL Injection) bilmek zorundayız.

Uniform Resource Locator (URL)

Bir web adresi olan **URL (Uniform Resource Locator)**, internet üzerindeki her türlü içeriğe (web sayfası, video, fotoğraf vb.) ulaşmamızı sağlayan bir rehberdir. Tarayıcımıza, gitmek istediğimiz kaynağın internetin neresinde olduğunu tam olarak tarif eder.

URL yapısını anlamak; sadece web'de gezinmek için değil, web uygulama geliştirme, hata ayıklama (troubleshooting) ve özellikle **siber güvenlik** (saldırı vektörlerini anlama) açısından kritiktir.



URL'in Anatomisi (Parçalarına Ayrılması)

Bir URL, her biri farklı bir göreve sahip olan birkaç temel bileşenden oluşur. Bu bileşenlerin nasıl bir araya geldiğini bilmek, bir saldırganın nereye manipüle edebileceğini anlamamızı sağlar.

1. Scheme (Protokol)

Scheme, web sitesine erişmek için kullanılan protokolü ifade eder. Tarayıcıya sunucuyla hangi dilde konuşması gerektiğini söyler.

- **HTTP (HyperText Transfer Protocol)**: Verinin şifrelenmeden iletildiği temel protokoldür.
- **HTTPS (Hypertext Transfer Protocol Secure)**: HTTP'nin güvenli halidir; bağlantıyı şifreler. Güvenlik uzmanları ve modern tarayıcılar, verilerin çalınmasını önlemek için her zaman HTTPS kullanımını zorunlu tutar (**HSTS** gibi mekanizmalarla).

2. User (Kullanıcı Bilgileri)

Bazı URL'ler, kimlik doğrulaması gerektiren kaynaklara erişmek için kullanıcı adı (ve bazen parola) içerebilir.

- **Format:** `http://username:password@example.com`
- **Güvenlik Riski:** Günümüzde bu yöntem neredeyse hiç kullanılmaz. Hassas bilgilerin URL içinde açık metin (clear-text) olarak görünmesi devasa bir güvenlik riskidir. Browser geçmişinde veya sunucu loglarında bu bilgilerin kalmasına neden olur.

3. Host / Domain (Ana Makine / Alan Adı)

URL'in en kritik parçasıdır; hangi web sitesine eriştiğinizi belirler. Her domain benzersizdir ve bir kayıt kuruluşu (registrar) aracılığıyla tescil edilir.

- **Siber Güvenlik Notu (Typosquatting)**: Saldırganlar, gerçek domain isimlerine çok benzeyen (örneğin `google.com` yerine `g00gle.com`) sahte isimler alırlar. Buna **Typosquatting** denir ve genellikle **Phishing** (Oltalama) saldırılarında kullanıcıyı kandırıp hassas bilgilerini çalmak için kullanılır.

4. Port (Port Numarası)

Port, tarayıcınızı web sunucusu üzerindeki doğru servise yönlendirir. Bunu, bir binadaki (sunucu) doğru daire kapısına (servis) gitmek gibi düşünebilirsiniz.

- **Aralık:** 1 ile 65.535 arasındadır.
- **Varsayılanlar:** HTTP için port **80**, HTTPS için port **443**'tür. Eğer URL'de port belirtilmezse tarayıcı protokole göre bu varsayılanları kullanır.

5. Path (Yol)

Sunucu üzerindeki spesifik bir dosya veya sayfayı işaret eder. Tarayıcı için bir yol haritası gibidir.

- **Önem:** Web uygulamaları, yetkisiz kullanıcıların hassas dosya yollarına (örneğin `/admin` veya `/config`) erişmesini engellemek için bu yolları güvene almalıdır.

6. Query String (Sorgu Dizisi)

URL içinde soru işaretinden (?) sonra gelen kısımdır. Genellikle arama terimleri veya form verilerini iletmek için kullanılır.

- **Örnek:** `search?q=cybersecurity`
- **Siber Güvenlik Notu (Injection):** Kullanıcılar query string kısımlarını manuel olarak değiştirebilir. Eğer uygulama bu veriyi temizlemeden (sanitization) işlerse, **SQL Injection** veya **Command Injection** gibi saldırılara karşı savunmasız kalır. Girdi her zaman "güvensiz" kabul edilmelidir.

7. Fragment (Bölüm/Fragman)

Diyez sembolü (#) ile başlar ve web sayfasının belirli bir bölümüne (bir başlık veya tablo gibi) doğrudan atlamayı sağlar.

- **Davranış:** Fragment kısmı genellikle sunucuya gönderilmez, tarayıcı tarafında işlenir.
- **Güvenlik Riski:** Query string'de olduğu gibi, buradaki veriler de manipüle edilebilir. Uygulama tarafında (özellikle JavaScript ile) bu veri kontrol edilmezse, **DOM-based XSS** gibi enjeksiyon saldırılarına kapı aralayabilir.

Özet Tablo: URL Bileşenleri

Bileşen	Örnek	Açıklama
Scheme	<code>https://</code>	Kullanılan protokol (Güvenlik için HTTPS tercih edilir).
Host	<code>tryhackme.com</code>	Erişilen ana makine/site adı.
Port	<code>:443</code>	İletişimin geçtiği kapı (HTTPS için 443).
Path	<code>/rooms</code>	Sunucudaki kaynağın konumu.
Query String	<code>?id=10</code>	Sunucuya iletilen değişken parametreler.
Fragment	<code>#task3</code>	Sayfa içindeki belirli bir çapa (anchor) noktası.

HTTP Messages (HTTP Mesajları)

HTTP mesajları, bir kullanıcı (**Client/İstemci**) ile web sunucusu (**Server**) arasında teatisi yapılan veri paketleridir. Web uygulamalarının nasıl çalıştığını anlamak için bu mesajları söküp takabilmek gerekir; çünkü kullanıcının talepleri ve sunucunun bu taleplere verdiği yanıtlar tamamen bu paketlerin içindedir.

İletişim iki yönlüdür:

1. **HTTP Requests (İstekler):** Kullanıcı tarafından, web uygulamasında bir eylemi tetiklemek için gönderilir.
2. **HTTP Responses (Yanıtlar):** Sunucu tarafından, gelen isteğe karşılık olarak gönderilir.

```
HTTP Request

POST /login HTTP/1.1
Host: tryhackme.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 43

username=aleksandra&password=securepassword
```

```
HTTP Response

HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 56
Date: Wed, 29 Aug 2024 12:00:00 GMT

{
  "message": "login successfull",
  "status": "success"
}
```

HTTP Mesajının Anatomisi (Yapısı)

Her HTTP mesajı (ister istek ister yanıt olsun), iletişimin hatasız yürümesi için belirli bir formatı takip eder. Bu yapı 4 ana bölümden oluşur:

1. Start Line (Başlangıç Satırı)

Mesajın "giriş" kısmıdır. Mesajın türünü ve nasıl işlenmesi gerektiğini belirten en kritik bilgileri içerir.

- **İsteklerde (Request):** Hangi **Metodun** (GET, POST vb.) kullanıldığını, hangi **URL/Path'e** gidildiğini ve **HTTP versiyonunu** belirtir.
- **Yanıtlarda (Response):** İşlemin sonucunu belirten **Status Code** (Durum Kodu, örn: 200 OK, 404 Not Found) bilgisini içerir.

2. Headers (Başlıklar)

"Anahtar-Değer" (**Key-Value**) çiftlerinden oluşan, mesaj hakkında ek bilgiler sağlayan kısımdır. Hem istemciye hem sunucuya talimatlar verir.

- **Neleri İçerir?** Güvenlik yapılandırmaları, içerik türü (**Content-Type**), çerezler (**Cookies**), tarayıcı bilgisi (**User-Agent**) gibi meta veriler burada yer alır.
- **Önem:** Tarayıcı ve sunucu arasındaki "anlaşmanın" detayları burada gizlidir. Örneğin, sunucu `Content-Type: application/json` diyerek gövdede JSON verisi gönderdiğini haber verir.

3. Empty Line (Boş Satır)

Başlıklar (Headers) ile mesaj gövdesi (Body) arasına giren, teknik olarak hayati bir bölücüdür.

- **Neden Var?** Sunucu veya tarayıcı, başlıkların nerede bittiğini ve asıl içeriğin nerede başladığını bu boş satır sayesinde anlar. Bu satır olmazsa veri yorumlanamaz ve iletişim hataları (Parsing Errors) oluşur.

4. Body (Gövde/Payload)

Asıl verinin taşındığı yerdir. Her mesajda bulunmak zorunda değildir (Örn: Basit bir GET isteğinde body genellikle boştur).

- **İsteklerde (Request Body):** Kullanıcının sunucuya gönderdiği verileri içerir (Örn: Giriş yaparken yazılan kullanıcı adı ve şifre gibi form verileri).
- **Yanıtlarda (Response Body):** Sunucunun kullanıcıya gönderdiği içeriktir (Örn: HTML sayfasının kodu, bir resim dosyası veya API'den gelen veriler).

Neden HTTP Mesajlarını Bilmeliyiz?

1. **Hata Ayıklama (Troubleshooting):** Web iletişimindeki aksaklıkları teşhis etmek için bu paketleri analiz ederiz. "Neden sayfa yüklenmiyor?" sorusunun cevabı genellikle hatalı bir header veya yanlış bir status kodundadır.
2. **Performans:** Mesajların doğru yapılandırılması (Örn: Caching header'ları), uygulamanın hızını doğrudan etkiler.
3. **Siber Güvenlik (Kritik):** * Veri iletimi sırasında güçlü güvenlik önlemleri (HSTS, CSP gibi header'lar) ancak bu mesaj yapısına hakim olunarak uygulanabilir.
 - Bir saldırgan olarak, bu mesajları **Burp Suite** gibi araçlarla yakalayıp (interception) manipüle ederek uygulamanın mantığını bozmaya çalışırız.

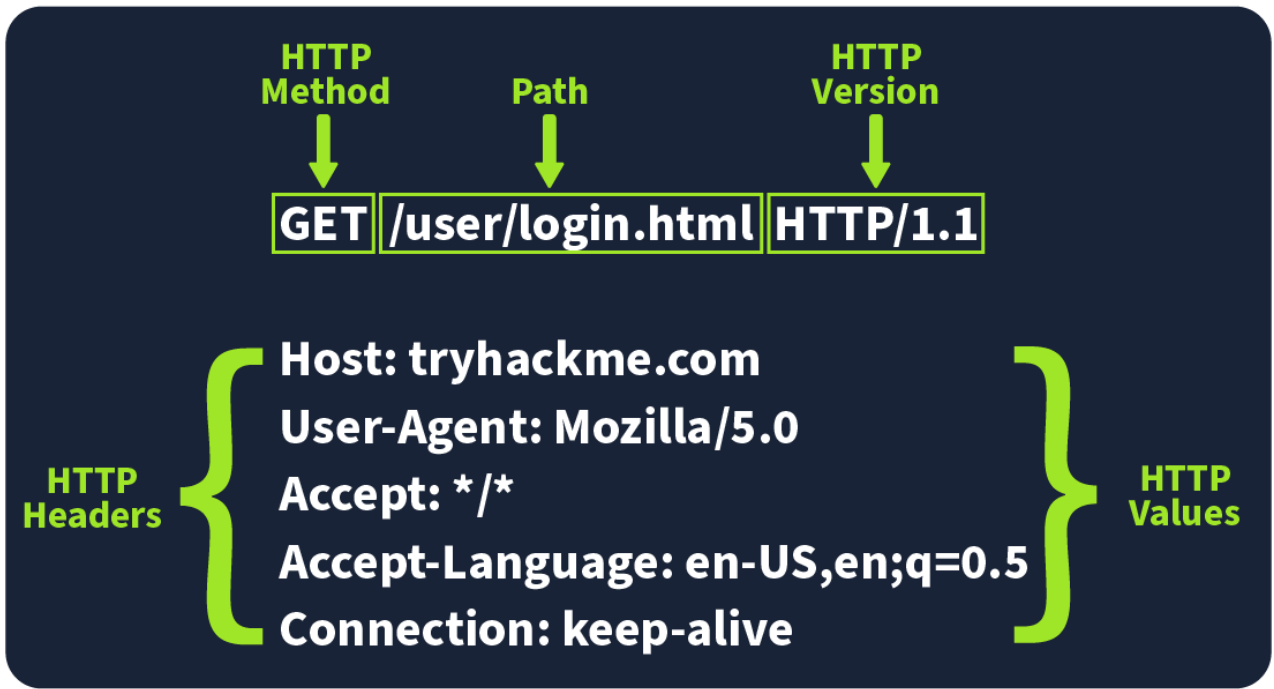
Bir HTTP Mesajı Örneği (Zihinde Canlandırma)

HTTP

GET /index.html HTTP/1.1	<-- Start Line (İstek Metodu, Yol, Versiyon)
Host: example.com	<-- Header (Hedef Sunucu)
User-Agent: Mozilla/5.0	<-- Header (Tarayıcı Bilgisi)
	<-- Empty Line (Boş Satır - AYIRICI)
(Body bu istekte boştur)	<-- Body

HTTP Request (Request Line ve Metotlar)

Bir **HTTP Request (İstek)**, kullanıcının web sunucusuyla etkileşime geçmek ve bir eylemi tetiklemek için gönderdiği pakettir. Siber güvenlik dünyasında, bir saldırganın sunucuyla ilk temas noktası burasıdır; bu yüzden yapısını atomlarına kadar ayırmak hayati önem taşır



1. Request Line (İstek Satırı)

İstek mesajının en üstündeki ilk satırdır. Sunucuya ne yapması gerektiğini söyleyen üç ana bileşenden oluşur:

- **Format:** METOT /yol HTTP/versiyon
- **Örnek:** GET /login HTTP/1.1

2. HTTP Metotları (Eylemler)

Metotlar, sunucuya URL ile belirtilen kaynak üzerinde hangi işlemin yapılacağını söyler. Güvenlik açısından her birinin kendine has riskleri vardır:

Sık Kullanılan Metotlar

- **GET:** Sunucudan veri **getirmek/okumak** için kullanılır. Sunucu üzerinde herhangi bir değişiklik yapmaz.
 - ⚠ **Güvenlik Uyarısı:** Hassas verileri (token, şifre vb.) asla GET ile gönderme! Çünkü bu veriler URL içinde açık metin (plaintext) olarak görünür, tarayıcı geçmişine ve sunucu loglarına kaydedilir.
- **POST:** Sunucuya veri **göndermek** (yeni bir kayıt oluşturmak veya güncellemek) için kullanılır.
 - ⚠ **Güvenlik Uyarısı:** Gelen veri her zaman kirli kabul edilmelidir. **SQL Injection** veya **XSS** saldırılarını önlemek için veri mutlaka doğrulanmalı ve temizlenmelidir (**Sanitization**).
- **PUT:** Sunucudaki bir kaynağı tamamen **değiştirmek** veya güncellemek için kullanılır.
 - ⚠ **Güvenlik Uyarısı:** Yetkisiz birinin dosya değiştirmesini önlemek için bu metot sıkı bir yetkilendirme (**Authorization**) kontrolüne tabi tutulmalıdır.
- **DELETE:** Belirtilen kaynağı sunucudan **siler**.

- **⚠ Güvenlik Uyarısı:** Sadece yetkili kullanıcıların erişimine açık olmalıdır; aksi halde veri kaybına yol açar.

Özel Durum Metotları

- **PATCH:** Kaynağın sadece bir kısmını günceller (PUT gibi tamamını değiştirmez). Küçük değişiklikler için verimlidir.
- **HEAD:** GET ile aynıdır ancak sunucu sadece **Header**'ları gönderir, mesaj gövdesini (içeriği) göndermez. Dosya boyutunu veya meta verileri kontrol etmek için hızlı bir yoldur.
- **OPTIONS:** Sunucunun o URL için hangi metotlara izin verdiğini sorgular. (Örn: "Bu dosyada DELETE yapabilir miyim?")
- **TRACE:** İsteğin sunucuya gidene kadar geçtiği yolu geri döndürür (Debug/Hata ayıklama için).
 - **⚠ Güvenlik Uyarısı:** Genellikle güvenlik riskleri nedeniyle (Cross-Site Tracing gibi) sunucularda kapatılması önerilir.
- **CONNECT:** HTTPS gibi şifreli iletişim için güvenli bir tünel oluşturur.

3. URL Path (Yol)

Sunucuda istenen kaynağın tam konumunu belirtir.

- **Örnek:** `https://tryhackme.com/api/users/123` adresinde yol `/api/users/123` kısmıdır.
- **Siber Güvenlik Bakışı:** Saldırganlar bu yolu manipüle ederek (Örn: **Directory Traversal** - Dizini yukarı doğru zorlama) yetkisiz dosyalara erişmeye çalışır.
- **Korunma:** Yol doğrulanmalı, temizlenmeli ve gizlilik risk değerlendirmesi yapılmalıdır.

4. HTTP Versiyonları

Protokolün zaman içindeki gelişimi hızı ve güvenliği artırmıştır:

Versiyon	Yıl	Özellikler
HTTP/0.9	1991	Sadece GET desteklerdi, başlıklar yoktu.
HTTP/1.0	1996	Header desteği geldi, farklı içerik tipleri taşınabilir oldu.
HTTP/1.1	1997	Halen yaygındır. Kalıcı bağlantılar (Keep-alive) ve gelişmiş önbellekleme geldi.
HTTP/2	2015	Çoklama (Multiplexing) ve header sıkıştırma ile hızlandı.
HTTP/3	2022	QUIC protokolünü kullanır; çok daha hızlı ve doğuştan daha güvenlidir.

Not: Modern sistemler hız için HTTP/2 veya HTTP/3'e geçse de, geriye dönük uyumluluk nedeniyle HTTP/1.1 her yerde karşımıza çıkar.

Önemli Hatırlatma: Bir siber güvenlik öğrencisi olarak, bir web sitesine sızmaya çalışırken ilk yapacağın işlerden biri `OPTIONS` metodu ile hangi kapıların (metotların) açık olduğunu kontrol etmektir.

HTTP Request - Headers (Başlıklar) ve Body (Gövde)

Bir HTTP isteğinin sadece ne yapmak istediğini (Metot) bilmek yetmez; bu eylemin detaylarını ve taşınan veriyi anlamak siber güvenlik analizi için asıl can alıcı noktadır. Bu bölümde isteğin "ayrıntılarını" (Headers) ve "yükünü" (Body) inceleyeceğiz.

1. Request Headers (İstek Başlıkları)

Başlıklar, web sunucusuna istek hakkında ek bilgiler ileten "anahtar-değer" çiftleridir. Sunucu, bu başlıklara bakarak isteği nasıl işleyeceğine karar verir.

En Sık Karşılaşılan İstek Başlıkları

Başlık (Header)	Örnek Kullanım	Açıklama
Host	Host: tryhackme.com	İsteğin hangi web sunucusuna (domain) yapıldığını belirtir. Aynı IP üzerinde birden fazla site varsa sunucu bu başlığa bakarak doğru siteyi açar.
User-Agent	User-Agent: Mozilla/5.0...	İsteği gönderen tarayıcı/yazılım bilgisini verir. Sunucu buna bakarak sayfayı mobil mi yoksa masaüstü mü göndereceğini seçer.
Referer	Referer: https://google.com/	Kullanıcının bu adrese hangi sayfadan gelerek (yönlendirilerek) ulaştığını gösterir.
Cookie	Cookie: session_id=abc123	Sunucunun tarayıcıya "hafızanda tut" dediği verilerdir. Oturum yönetimi ve kullanıcı takibi için kritiktir.
Content-Type	Content-Type: application/json	İstek gövdesinde (Body) hangi formatta veri gönderildiğini sunucuya haber verir.

2. Request Body (İstek Gövdesi)

POST ve **PUT** gibi sunucuya veri gönderilen metotlarda, asıl veri paketinin (payload) bulunduğu kısımdır. Verinin formatı sunucuya `Content-Type` başlığı ile bildirilir. İşte en yaygın veri formatları:

A. URL Encoded (`application/x-www-form-urlencoded`)

Klasik HTML formlarının varsayılan formatıdır. Veriler `anahtar=değer` çiftleri halinde dizilir ve çiftler ampersand (`&`) ile ayrılır. Boşluk gibi özel karakterler yüzde kodlamasıyla (percent-encoding) gönderilir.

- **Örnek Yapı:**

HTTP

```
POST /profile HTTP/1.1
Host: tryhackme.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 33

name=Aleksandra&age=27&country=US
```

Not: Buradaki `Content-Length` başlığı, body'deki verinin kaç karakter (bayt) olduğunu söyler. Eksik yazılırsa veri kesilebilir.

B. Form Data (`multipart/form-data`)

Dosya yükleme (upload) işlemlerinde veya büyük veri blokları gönderilirken kullanılır. Her bir veri parçası bir **boundary** (sınır) dizesi ile ayrılır.

- **Örnek Yapı:**

HTTP

```
POST /upload HTTP/1.1
Content-Type: multipart/form-data; boundary=----WebKitFormBoundary7MA4W

----WebKitFormBoundary7MA4W
Content-Disposition: form-data; name="username"

aleksandra
----WebKitFormBoundary7MA4W
Content-Disposition: form-data; name="profile_pic"; filename="pic.jpg"
Content-Type: image/jpeg

[Burada resmin ikili/binary verisi bulunur]
----WebKitFormBoundary7MA4W--
```

C. JSON (`application/json`)

Modern API'lerin ve web uygulamalarının en çok tercih ettiği formattır. Veri süslü parantez { } içinde, `isim : değer` çiftleri halinde taşınır.

- **Örnek Yapı:**

JSON

```
{
  "name": "Aleksandra",
  "age": 27,
  "country": "US"
}
```

D. XML (application/xml)

Daha eski ama hala kurumsal sistemlerde kullanılan etiket tabanlı bir yapıdır. Veriler `<tag>` ve `</tag>` işaretleri arasına yazılır.

- **Örnek Yapı:**

XML

```
<user>
  <name>Aleksandra</name>
  <age>27</age>
</user>
```

Siber Güvenlik İçin Kritik Notlar

1. **Manipülasyon:** Bir web uygulamasını test ederken (Pentest), `Content-Type` başlığını değiştirerek sunucunun nasıl tepki verdiğini ölçebilirsin. Örneğin sunucu sadece JSON bekliyorsa ama sen XML gönderiyorsan, hata mesajlarından sunucu hakkında bilgi sızdırabilirsin.
2. **Cookie Güvenliği:** `Cookie` başlığındaki veriler asla şifrelenmemiş halde kullanıcı parolası taşımamalıdır. Buradaki veriler çalınırsa "Session Hijacking" (Oturum Çalma) saldırısı gerçekleşir.
3. **Girdi Doğrulama:** Body içindeki her format (JSON, XML vb.) kendi içinde özel saldırılara gebedir. Örneğin XML formatı **XXE (XML External Entity)** saldırılarına, JSON ise yanlış parse edilirse enjeksiyonlara neden olabilir.

HTTP Response (Status Line ve Durum Kodları)

Kullanıcı bir istek (Request) gönderdiğinde, sunucu bu isteği işler ve bir **HTTP Response (Yanıt)** döndürür. Bu yanıt, işlemin başarılı olup olmadığını veya bir sorun çıktıysa bunun ne olduğunu anlamamızı sağlayan bir "durum raporu" niteliğindedir.



1. Status Line (Durum Satırı)

Her HTTP yanıtının ilk satırıdır. Sunucunun verdiği cevabın "özetini" buradadır ve üç parçadan oluşur:

- **HTTP Versiyonu:** (Örn: HTTP/1.1) Kullanılan protokol sürümü.
- **Status Code (Durum Kodu):** İşlemin sonucunu belirten 3 haneli sayı.
- **Reason Phrase (Neden İfadesi):** Durum kodunun insan tarafından okunabilir, kısa açıklaması (Örn: OK , Not Found).

Örnek bir Status Line: HTTP/1.1 200 OK

2. Durum Kodları ve Kategorileri

Durum kodları, ilk rakamlarına göre 5 ana kategoriye ayrılır. Bu kategorizasyon, bir hatanın istemci kaynaklı mı yoksa sunucu kaynaklı mı olduğunu sanjeler içinde anlamamızı sağlar:

1xx: Bilgilendirme Yanıtları (Informational)

Sunucunun isteğinin ilk kısmını aldığını ve işlemin devam ettiğini belirtir. Bir nevi "devam et" sinyalidir.

- **100 (Continue):** Sunucu isteğinin başlangıcını aldı, geri kalanını bekliyor.

2xx: Başarı Yanıtları (Successful)

İsteğinin sunucu tarafından başarıyla alındığını, anlaşıldığını ve kabul edildiğini gösterir. En çok görmek istediğimiz kategoridir.

- **200 (OK):** Her şey yolunda. İstenen veri paket içerisinde gönderiliyor.

3xx: Yönlendirme Mesajları (Redirection)

İstenen kaynağın yerinin değiştiğini ve tarayıcının yeni bir adrese gitmesi gerektiğini bildirir.

- **301 (Moved Permanently):** Kaynak kalıcı olarak yeni bir URL'e taşındı. Gelecekteki tüm istekler yeni adrese yapılmalıdır.

4xx: İstemci Hatası Yanıtları (Client Error)

Sorun gönderilen istektedir. URL yanlış olabilir, yetki eksik olabilir veya format hatalıdır.

Siber güvenlikte en çok bu kodları analiz ederiz (Örn: Gizli dizinleri ararken aldığımız 403 veya 404 yanıtları).

- **404 (Not Found):** Sunucu belirtilen URL'de hiçbir şey bulamadı. Adresi kontrol et!
- **403 (Forbidden):** Kaynak orada var ama senin oraya girmeye yetkin yok.

5xx: Sunucu Hatası Yanıtları (Server Error)

İstek doğru olabilir ama sunucu tarafında bir şeyler patlamıştır (Veritabanı çökmesi, kod hatası vb.).

- **500 (Internal Server Error):** Sunucu tarafında beklenmedik bir hata oluştu ve istek işlenemedi.

En Sık Karşılaşılan Durum Kodları Özeti

Kod	İfade	Anlamı ve Siber Güvenlik Notu
200	OK	İstek başarılı. Hedef sayfa ele geçirildi.
301	Moved Permanently	Sayfa taşınmış. Yeni adresi Location header'ında bulabilirsin.
401	Unauthorized	Kimlik doğrulaması (Login) gerekiyor.
403	Forbidden	Erişim yasak. WAF veya dosya izinleri seni engelliyor olabilir.
404	Not Found	Dosya yok. Dizin tarama (Directory Brute Force) yaparken bu kodları filtreleriz.
500	Internal Server Error	Sunucu tarafında bir zafiyet tetiklenmiş olabilir (Örn: SQL Injection sonrası sunucu hatası).

Neden Bu Kodları Bilmeliyiz?

Bir sızma testi (Pentest) sırasında, bir aracı (örneğin **ffuf** veya **Gobuster**) kullanarak binlerce istek göndeririz. Eğer sunucu bize sürekli **404** döndürüyorsa aradığımız dosyalar orada değildir. Ancak aniden bir **200** veya **403** alırsak, orada ilginç bir şeyler (yönetim paneli, konfigürasyon dosyası vb.) olduğunu anlarız.

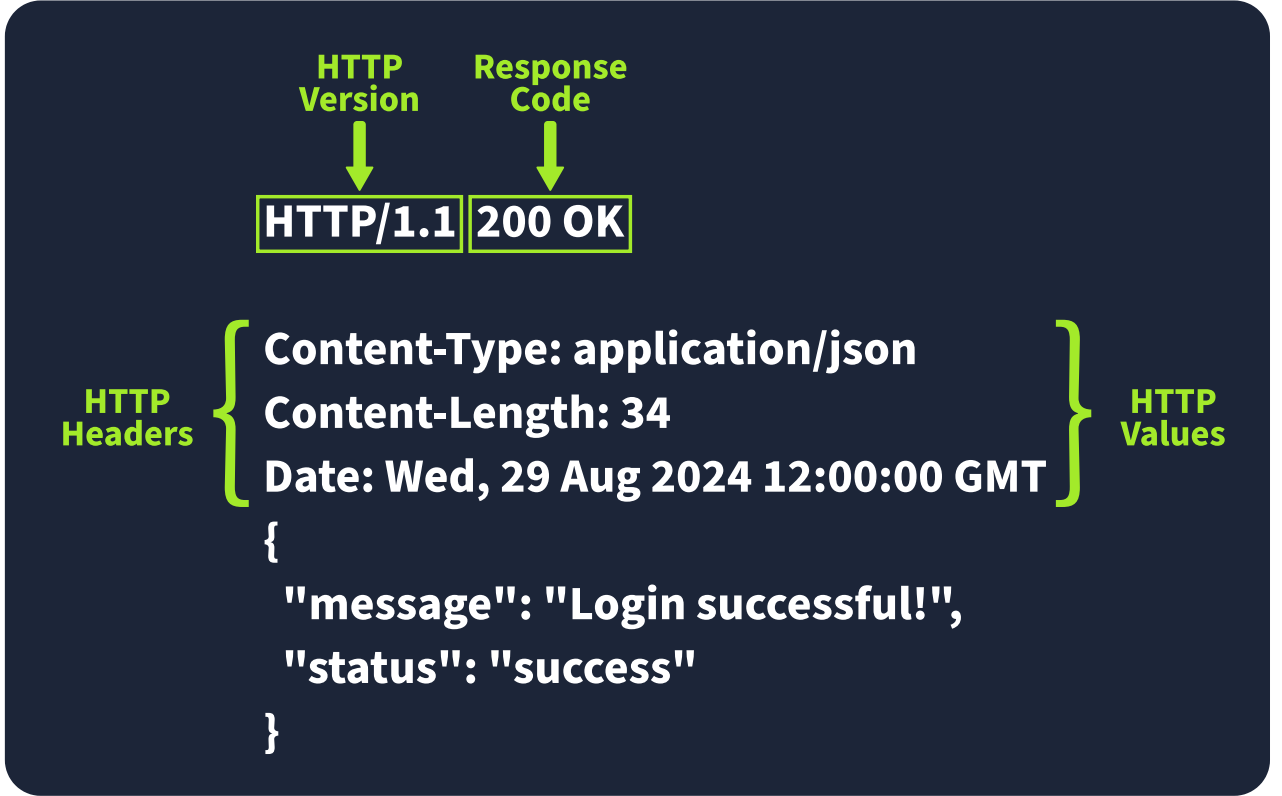
Kritik İpucu: 500 hataları her zaman kötü değildir; bazen bir giriş alanına gönderdiğimiz özel karakterlerin sunucu tarafındaki kodu bozduğunu ve orada bir **Injection** zafiyeti olabileceğini fısıldar.

HTTP Response - Headers (Başlıklar) ve Body (Gövde)

Bir sunucu isteğimize yanıt verdiğinde, sadece bir durum kodu (200, 404 vb.) göndermez; aynı zamanda tarayıcının bu yanıtı nasıl işlemesi gerektiğini söyleyen **Response Headers** (Yanıt Başlıkları) ve asıl içeriği barındıran **Response Body** (Yanıt Gövdesi) kısımlarını da gönderir.

1. Response Headers (Yanıt Başlıkları)

Yanıt başlıkları, sunucudan istemciye (tarayıcıya) gönderilen "anahtar-değer" çiftleridir. Sunucu hakkında bilgi verir ve güvenliği yapılandırır.



Kritik ve Zorunlu Başlıklar

İletişimin sağlıklı yürümesi için sunucunun genellikle dahil ettiği temel başlıklar şunlardır:

- **Date:** Yanıtın sunucu tarafından oluşturulduğu tam tarih ve saati belirtir. (Örn: Date: Fri, 23 Aug 2024 10:43:21 GMT)
- **Content-Type:** Tarayıcıya gelen verinin formatını söyler. Eğer bu text/html ise tarayıcı bunu bir web sayfası olarak çizer; application/json ise veri olarak işler. Karakter setini (UTF-8 gibi) de burada belirtir.
- **Server:** Sunucuda hangi yazılımın (Apache, Nginx, Microsoft-IIS) çalıştığını gösterir.
 - **⚠ Siber Güvenlik Notu:** Bu başlık saldırganlar için bir bilgi toplama (**Information Gathering**) kaynağıdır. Saldırgan, sunucu sürümünü öğrenirse o sürüme ait bilinen zafiyetleri (Exploit) araştırabilir. Bu yüzden birçok sistem yöneticisi bu başlığı gizler veya yanıltıcı bilgi yazar.

Diğer Yaygın ve Güvenlik Odaklı Başlıklar

- **Set-Cookie:** Sunucunun tarayıcıya "Bu veriyi kaydet ve sonraki isteklerde bana geri gönder" dediği başlıktır. Genellikle **Session ID** (Oturum Kimliği) taşır.
 - **Güvenlik Sıkılaştırma:** Çerezlerin çalınmasını önlemek için `HttpOnly` (JavaScript erişimini engeller) ve `Secure` (Sadece HTTPS üzerinden gönderilmesini sağlar) flag'leri eklenmelidir.
- **Cache-Control:** İçeriğin tarayıcı önbelleğinde ne kadar süre saklanacağını belirler. (Örn: `max-age=600` yani 10 dakika). Hassas verilerde `no-cache` kullanılarak verinin diske kaydedilmesi önlenir.
- **Location:** 3xx (Yönlendirme) kodlarıyla birlikte kullanılır. Tarayıcıya gideceği yeni adresi söyler. (Örn: `Location: /login.html`)
 - **⚠ Siber Güvenlik Notu (Open Redirect):** Eğer uygulama, kullanıcıdan aldığı bir girdiyi kontrol etmeden `Location` başlığına yerleştirirse, saldırgan kullanıcıyı kendi zararlı sitesine yönlendirebilir.

2. Response Body (Yanıt Gövdesi)

Sunucunun kullanıcıya gönderdiği asıl "paket" buradadır. İstedığımız her şey (HTML kodları, resimler, video dosyaları, API'den gelen JSON verileri) burada yer alır.

- **HTML:** Sayfanın görsel yapısı.
- **JSON/XML:** Uygulamalar arası veri transferi.
- **Binary:** Resim, PDF veya indirilebilir dosyalar.

Güvenlik Hatırlatması (XSS Önleme)

Sunucu, yanıt gövdesine veri yerleştirirken (özellikle kullanıcıdan gelen yorumlar, isimler vb.) bu veriyi mutlaka **sanitize** etmeli (temizlemeli) ve **escape** etmelidir (özel karakterleri etkisiz hale getirmelidir). Aksi takdirde, bir saldırganın gönderdiği `<script>` bloğu başka bir kullanıcının tarayıcısında çalışabilir; buna **Cross-Site Scripting (XSS)** denir.

Özet: Bir Yanıtın Anatomisi

HTTP

```
HTTP/1.1 200 OK          <-- Status Line
Date: Fri, 23 Aug 2024 10:43:21 GMT  <-- Header
Server: Apache/2.4.41 (Ubuntu)      <-- Header (Bilgi Sızdırıyor olabilir!)
Content-Type: text/html; charset=UTF-8  <-- Header
Content-Length: 156             <-- Header
                                <-- Boş Satır (Ayrıcı)
<html>                          <-- Body (Başlangıç)
  <body>
    <h1>Merhaba Dünya!</h1>
```



```
</body>  
</html>
```

```
<-- Body (Bitiş)
```

Önemli Notlar:

- **Status Code 301/302 görürsen:** Gözün hemen Location header'ını arasın, çünkü yeni adres orada.
- **Pentest yapıyorsan:** Server header'ına bakıp versiyon numarası ara, CVE (Common Vulnerabilities and Exposures) listelerini kontrol et.
- **Oturum çalmaya çalışıyorsan:** Set-Cookie header'ında HttpOnly flag'i var mı bak. Yoksa JavaScript (XSS) ile o çerezi çalabilirsin demektir.

Security Headers (Güvenlik Başlıkları)

Security Headers, bir web uygulamasının genel güvenlik seviyesini artıran özel HTTP yanıt başlıklarıdır. Bu başlıklar; **Cross-Site Scripting (XSS)**, **Clickjacking** ve diğer yaygın saldırı türlerine karşı sunucu seviyesinde bir savunma hattı (mitigation) oluşturur.

Bir web sitesinin güvenlik başlıklarını analiz etmek için <https://securityheaders.io> gibi araçlar kullanılır. Şimdi, modern web güvenliğinin "Dört Atlısı" diyebileceğimiz kritik başlıkları detaylandıralım:

1. Content-Security-Policy (CSP)

CSP, tarayıcıya "Hangi kaynaklara güvenmelisin?" talimatını veren ek bir güvenlik katmanıdır. Saldırganın siteye enjekte ettiği kötü niyetli script'lerin (XSS) çalışmasını engellemek için kullanılır.

- **Mantık:** Yönetici, hangi domain'lerden veya kaynaklardan veri çekilebileceğini beyaz listeye (whitelist) alır.
- **Kritik Kelime - 'self' :** Sadece web sitesinin kendi domain'inden içerik yüklenmesine izin verildiğini belirtir.

Örnek CSP Analizi:

HTTP

```
Content-Security-Policy: default-src 'self'; script-src 'self'  
https://cdn.tryhackme.com; style-src 'self'
```

- **default-src 'self':** Varsayılan olarak her şey (resim, font vb.) sadece sitenin kendi adresinden yüklenebilir.
- **script-src 'self' <https://cdn.tryhackme.com>:** JavaScript dosyaları hem kendi sitemizden hem de sadece <https://cdn.tryhackme.com> adresinden çekilebilir. Başka bir yerden gelen script'ler tarayıcı tarafından bloklanır.

- **style-src 'self'**: CSS dosyaları sadece sitemizden yüklenebilir.

2. Strict-Transport-Security (HSTS)

HSTS, tarayıcının o siteyle **her zaman** HTTPS (şifreli) üzerinden konuşmasını zorunlu kılar. Bu, "Man-in-the-Middle" (Aradaki Adam) saldırılarını ve protokol düşürme (SSL Stripping) girişimlerini engeller.

Örnek HSTS Analizi:

HTTP

```
Strict-Transport-Security: max-age=63072000; includeSubDomains; preload
```

- **max-age**: Bu ayarın saniye cinsinden ne kadar süre (burada 2 yıl) geçerli olacağını belirtir.
- **includeSubDomains**: Bu kuralın tüm alt alan adları (örn: `blog.site.com`, `mail.site.com`) için de geçerli olmasını sağlar.
- **preload**: Sitenin tarayıcıların "ön yükleme listesine" dahil edilmesini sağlar. Böylece kullanıcı siteyi hayatında ilk kez ziyaret ediyor olsa bile tarayıcı doğrudan HTTPS ile bağlanır.

3. X-Content-Type-Options

Tarayıcıların "MIME-type sniffing" (dosya türünü tahmin etme) özelliğini kontrol eder. Normalde tarayıcılar, bir dosyanın içeriğine bakıp onun ne olduğunu tahmin etmeye çalışır. Saldırganlar bunu kullanarak, zararlı bir script'i masum bir resim dosyasıymış gibi (`.jpg`) sunucuya yükleyip tarayıcıyı kandırabilir.

Örnek Kullanım:

HTTP

```
X-Content-Type-Options: nosniff
```

- **nosniff**: Tarayıcıya "Tahmin yürütme! Sadece `Content-Type` başlığında ne yazıyorsa onu kullan" der. Eğer sunucu `Content-Type: text/plain` diyorsa, tarayıcı o dosyayı asla bir script olarak çalıştırmaz.

4. Referrer-Policy

Bir kullanıcı bir linke tıkladığında, tarayıcı gittiği yeni siteye "nereden geldiğini" (`Referer` başlığı ile) söyler. Bu başlık, hangi bilgilerin paylaşılacağını kontrol ederek gizliliği korur.

Yaygın Direktifler:

- **no-referrer**: Hiçbir yönlendirme bilgisi gönderilmez.
- **same-origin**: Sadece site içindeki linklere tıklanıldığında bilgi gönderilir; harici sitelere bilgi gitmez.
- **strict-origin**: Sadece protokol güvenliği korunduğunda (HTTPS'den HTTPS'e) kaynak bilgisi gönderilir.
- **strict-origin-when-cross-origin**: Site içi dolaşımda tam URL gönderilirken, başka bir HTTPS sitesine gidildiğinde sadece ana domain bilgisi gönderilir.

Güvenlik Notları Özet Tablosu

Güvenlik Başlığı	Engellediği Başlıca Saldırı
CSP	Cross-Site Scripting (XSS), Data Injection
HSTS	SSL Stripping, MITM (Aradaki Adam)
X-Content-Type-Options	MIME Sniffing, Polyglot Attack
Referrer-Policy	Information Leakage (Bilgi Sızıntısı)

Not: Bir sızma testine başlarken hedef sitenin `securityheaders.io` raporuna bakmak, defanstaki boşlukları görmeni sağlar. Eğer **CSP** eksikse, ilk deneyeceğin şey bir **XSS** zafiyeti aramak olmalıdır.